

TRABAJO PRACTICO INGENIERIA DE SOFTWARE



Nombre del sistema: Supermercado lugo

Apellido y Nombre de alumnos: Lucas Aguilar y
Thiago Ragusa

Localización: Castelar

Comisión: 3-A

Turno: Noche

Año de cursada: 3er año

Apellido y Nombre del docente: Julián Martín
Rodríguez

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				Versión: 1.0

Índice

3.....	Historial de revisiones/Arquitectura básica
4.....	Arquitectura básica
5.....	Log in y Log out objetivos / diagrama de secuencia (Log in)
7.....	Diagrama de secuencia (Log out)
9.....	Manejo de usuarios objetivo / diagrama de clases
10.....	Diagrama de secuencia (Agregar)
12.....	Diagrama de secuencia (Modificar)
14.....	Diagrama de secuencia (Borrar)
16.....	Criptografía
17.....	Composite/manejo de usuario
19.....	Diagrama de secuencia (Agregar permiso)
21.....	Diagrama de secuencia (Eliminar permiso)
23.....	Observer/Multi-Idioma
24.....	Diagrama de secuencia (Agregar Lenguaje)
26.....	Diagrama de secuencia (Crear Traducción)
28.....	Bitácora
29.....	Control de cambios
30.....	Digito Verificador

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez	
	Alumno: Lucas Aguilar, Thiago Ragusa		Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3
	<Nombre del Sistema>			Versión: 1.0
	<Nombre de Sección / Etapa>			

HISTORIAL DE REVISIONES

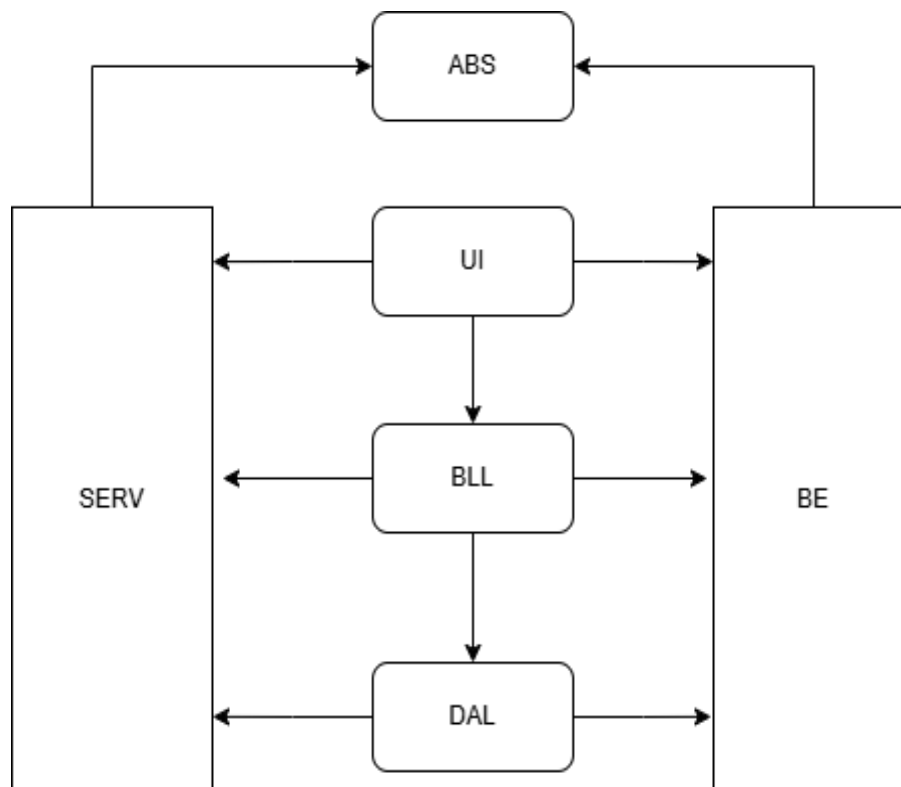
Fecha	Versión	Autor	Descripción
9/5/25	1.0	Lucas Aguilar y Thiago Ragusa	Primer Entrega
13/6/25	2.0	Lucas Aguilar y Thiago Ragusa	Segunda entrega
11/7/25	3.0	Lucas Aguilar y Thiago Ragusa	Tercera Entrega

ARQUITECTURA BASICA

Objetivo

El objetivo que se intenta alcanzar con la arquitectura básica que se usa en el proyecto es mayoritariamente el de una organización bien estructurada, razón por la cual se ha elegido la arquitectura de N capas, la cual permite dividir el sistema en capas lógicas, con cada capa teniendo un objetivo que cumplir en particular y pudiendo recibir apoyo y ayuda de las demás, además de que se facilita el mantenimiento, actualización y expansión del sistema, con los cambios de cada capa siendo propios a esta y no cambiando todo el código en general (aunque si afectándolo de cierta manera)

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				Versión: 1.0



Como se puede ver en la representación gráfica, la arquitectura está compuesta por 6 capas, siendo estas la capa de UI (la presentación, donde se tiene la interfaz física con la cual interactuara el cliente/usuario del sistema, mostrándose datos en GUI y computando los inputs del usuario), la BLL (que se encarga de contener la lógica del dominio/reglas de negocio del proyecto, exponiendo sus servicios. Contiene los métodos de las clases), la DAL (que se encarga de la persistencia de los datos, usando para ello una base de datos, en este caso SQL), BE (medio de transporte para llevar los atributos de las clases entre capas), Servicio (se encarga de manejar lo relacionado a la verificación del log in y log out del usuario, aparte de las sesiones

en el sistema) y Abstracción (se encarga de entregar interfaces para que sean usadas, en este caso, en la capa de servicio, conteniendo una interfaz usuario que tiene la firma de los atributos que debería tener un usuario)

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez	
	Alumno: Lucas Aguilar, Thiago Ragusa		Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3
	<Nombre del Sistema>			Versión: 1.0
	<Nombre de Sección / Etapa>			

LOG IN Y LOG OUT

Objetivo

El objetivo que se espera cumplir con la inclusión del log in y log out es el de permitirle a un usuario ingresar al sistema y hacer uso de sus componentes, primero registrándose en una pantalla donde escribe ciertos datos (vistos en la parte de “control de usuario”) y posteriormente recibiendo el permiso de ingresar al sistema si es que se colocaron los datos correctos y elementos del sistema como la capa de servicio fueron capaces de verificar que el usuario ya se encuentra como parte de una sesión existente

Diagrama de secuencia y especificación (Log in)

Nombre del Caso de uso: CU001-Inicio de Sesión
Actores: Usuario
Flujo Principal: 1-El usuario llena los textbox con sus datos: - a-Nombre - b-Contraseña 2-El usuario aprieta el botón Login 3-El sistema verifica que los textboxes estén llenos 4-El sistema encripta el nombre y contraseña del usuario 5-El sistema verifica que no haya otra sesión iniciada 6-El sistema verifica que la información sea correcta - a- Nombre y Contraseña en lista de usuarios 7-El sistema crea el usuario y hace el Log in 8-El sistema abre el form “MenúSelección”, cerrando el de
log in
Flujo Alternativo:

manda

3.1-El sistema detecta que los textboxes están vacíos y

Un aviso al usuario

3.2-El sistema retorna al punto 1 del flujo principal

5.1-El sistema detecta que hay otra sesión ya iniciada y

Manda un aviso

5.2-El sistema retorna al punto 1 del flujo principal

6.1-El sistema detecta que el usuario es incorrecto y

Manda un aviso

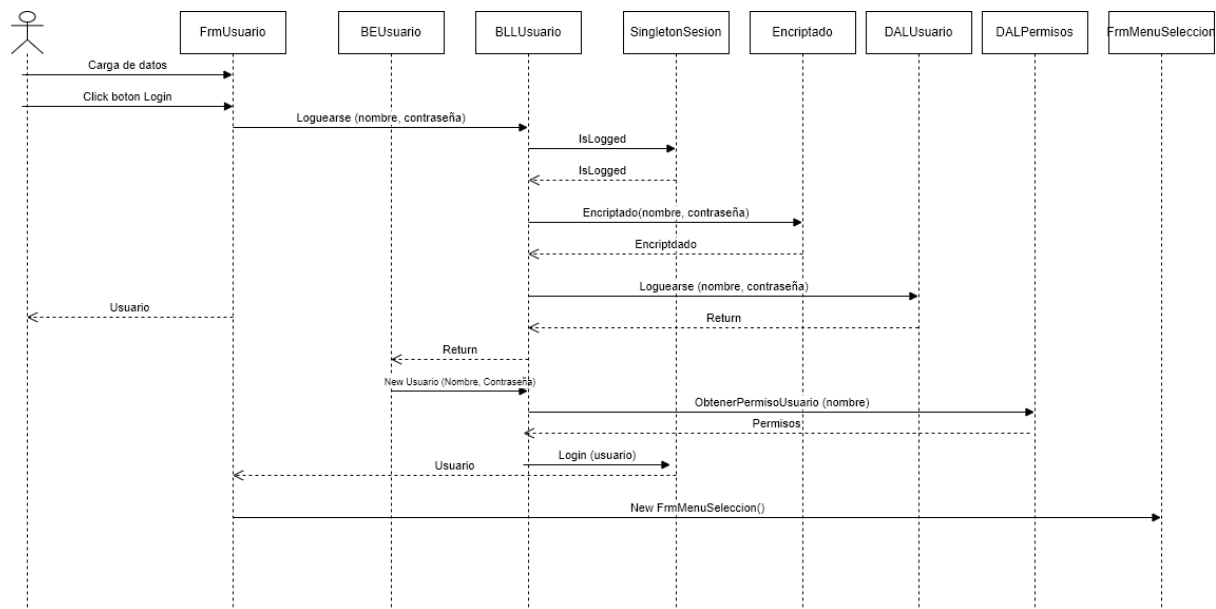
6.2- El sistema retorna al punto 1 del flujo principal

Descripción: Se inicia una sesión con un cierto usuario

Precondiciones: Que el usuario haya entrado a la pagina

Postcondiciones: Se Inicia la sesión y se muestra el form principal

Puntos de extensión: Ninguno

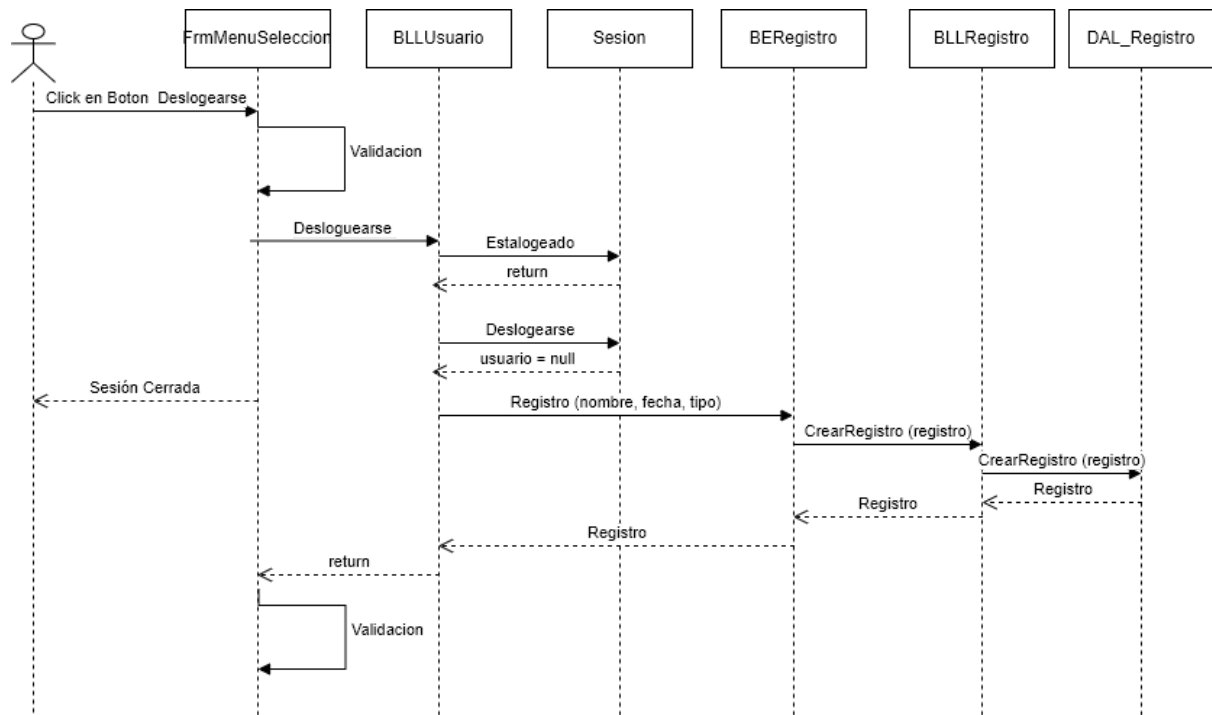


UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>			Versión: 1.0	
	<Nombre de Sección / Etapa>				

Diagrama de secuencia y especificación (Log out)

Nombre del Caso de uso: CU002-Cierre de sesión	
Actores: Usuario	
Flujo Principal:	1-El usuario aprieta el botón “Cerrar Sesión” en form “MenúSelección” 2-El sistema verifica que haya una sesión iniciada 3-El sistema cierra la sesión del usuario 4-El sistema muestra un mensaje de confirmación de cierre
Flujo Alternativo:	2.1-El sistema detecta que no hay una sesión iniciada y avisa 2.2-El sistema retorna al punto 1 del flujo principal
Descripción: Se cierra la sesión del usuario	
Precondiciones: Que el usuario haya iniciado la sesión anteriormente	
Postcondiciones: Se cierra la sesión y se vuelve a la pantalla principal	
Puntos de extensión: Ninguno	

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez	
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3
	<Nombre del Sistema>			
	<Nombre de Sección / Etapa>			
				Fecha: 15/4/25
				Versión: 1.0

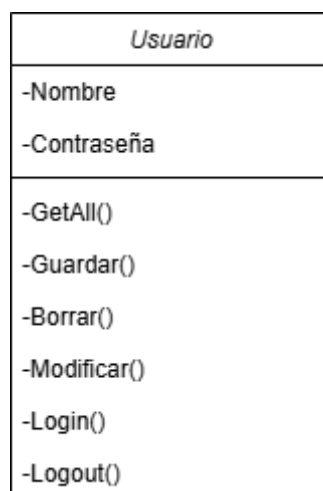


UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				Versión: 1.0

MANEJO DE USUARIOS (ABM)

Objetivo

El objetivo que se quiere conseguir con el manejo de usuarios es permitirle a el usuario que utiliza el sistema tener acceso a una lista de todos los usuarios registrados en este, con cada usuario teniendo una serie de métodos y 2 atributos en particular únicos para cada uno (Nombre y Contraseña), como puede verse en el siguiente diagrama de clases



Con esto, podemos entonces realizar las operaciones de Guardar, borrar y modificar a los usuarios en la lista, los cuales se encuentra en la base de datos, pudiendo así

controlar la información de estos y cuantos se encuentran registrados en nuestro sistema, como se puede ver a continuación

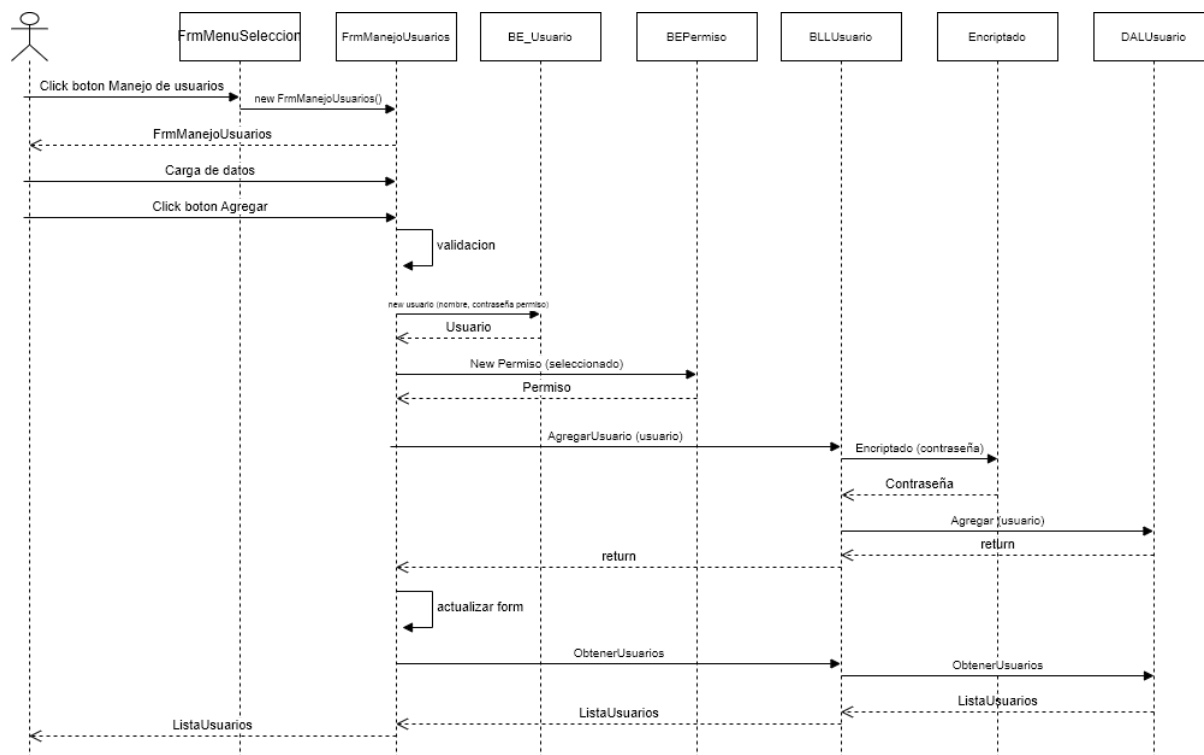
UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez	
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:
	Localización:	Comisión:	Turno:	Año:
	Castelar	3-A	Noche	3
	<Nombre del Sistema>			
	<Nombre de Sección / Etapa>			
				Fecha: 15/4/25
				Versión: 1.0

Diagrama de secuencia y especificación (Agregar)

Nombre del Caso de uso: CU003-Agregar Usuario	
Actores: Usuario	
Flujo Principal:	1-El usuario aprieta el botón “Manejo de usuarios” en el form “MenúSeleccion” 2-El sistema despliega el form “Manejo Usuarios” 3-El usuario introduce los datos solicitados a-Nombre b-Contraseña 4-El usuario aprieta el botón “Agregar” 5-El sistema verifica que los datos sean correctos A-Nombre y contraseña > 4 caracteres 6-El sistema crea el usuario con los datos entregados y sus permisos 7-El sistema encripta los datos del usuario 8-El sistema encripta los datos y agrega al usuario a la lista de usuarios 9-El sistema muestra la lista de usuarios actualizada
Flujo Alternativo:	5.1-El sistema detecta que hay menos de 4 caracteres 5.2-El sistema retorna al punto 3 del flujo principal
Descripción: Se agrega un usuario a la lista en la base de datos	

Precondiciones: Que el usuario haya iniciado sesión de forma correcta y tenga permisos
Postcondiciones: Se agrega el usuario a la lista de usuarios
Puntos de extensión: Ninguno

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización:	Comisión:	Turno:	Año:	
	Castelar	3-A	Noche	3	
	<Nombre del Sistema>				Versión: 1.0
	<Nombre de Sección / Etapa>				



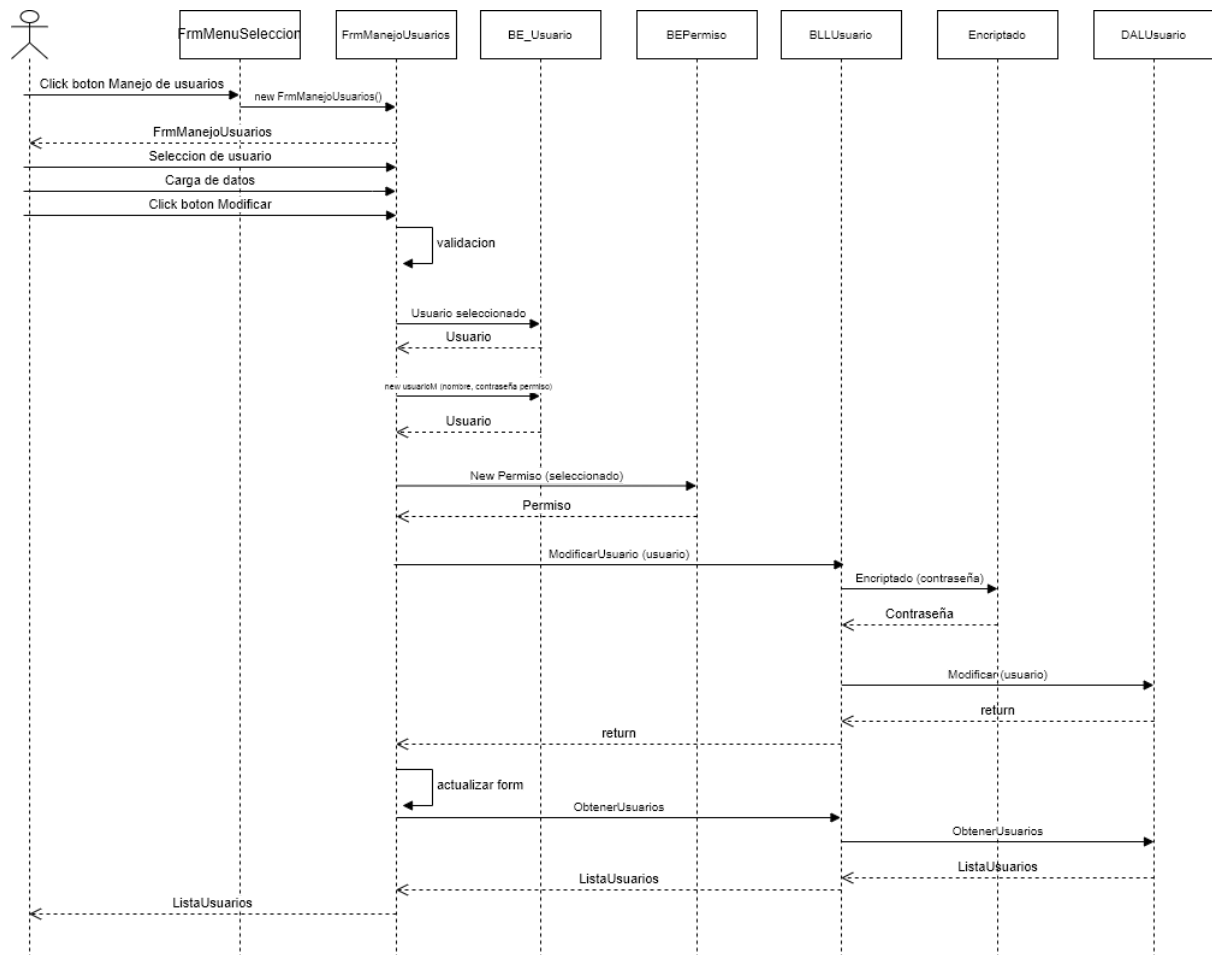
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				Versión: 1.0
	<Nombre de Sección / Etapa>				

Diagrama de secuencia y especificación (Modificar)

Nombre del Caso de uso: CU004-Modificar Usuario	
Actores: Usuario	
Flujo Principal:	1-El usuario aprieta el botón “Manejo de usuarios” en el form “MenúSeleccion” 2-El sistema despliega el form “Manejo Usuarios” 3-El usuario hace click en el usuario a modificar en la lista 4- El usuario introduce los datos solicitados a-Nombre b-Contraseña 5- El usuario aprieta el botón “Modificar” 6- El sistema verifica que los datos sean correctos A-Nombre y contraseña > 4 caracteres 7-El sistema crea un usuario seleccionado con los datos del usuario al que se le hizo click 8-El sistema crea un nuevo usuario con la ID del seleccionado y los datos modificados en los textboxes 9-El sistema encripta los datos y agrega el usuario modificado a la lista de usuarios 10-El sistema muestra la lista de usuarios actualizada
Flujo Alternativo:	6.1-El sistema detecta que hay menos de 4 caracteres 6.2-El sistema retorna al punto 3 del flujo principal
Descripción: Se modifica un usuario seleccionado de la lista de usuarios	
Precondiciones: Que el usuario haya iniciado sesión de forma correcta y tenga permisos	
Postcondiciones: Se modifica el usuario seleccionado y se muestra en la lista de usuarios	
Puntos de extensión: Ninguno	

UNIVERSIDAD ABIERTA INTERAMERICANA			
Facultad de Tecnología Informática			
	Materia: Ingeniería de software	Docente: Julián Martín Rodríguez	Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa	Legajo:	

	Localización:	Comisión:	Turno:	Año:	Versión: 1.0
	Castelar	3-A	Noche	3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				

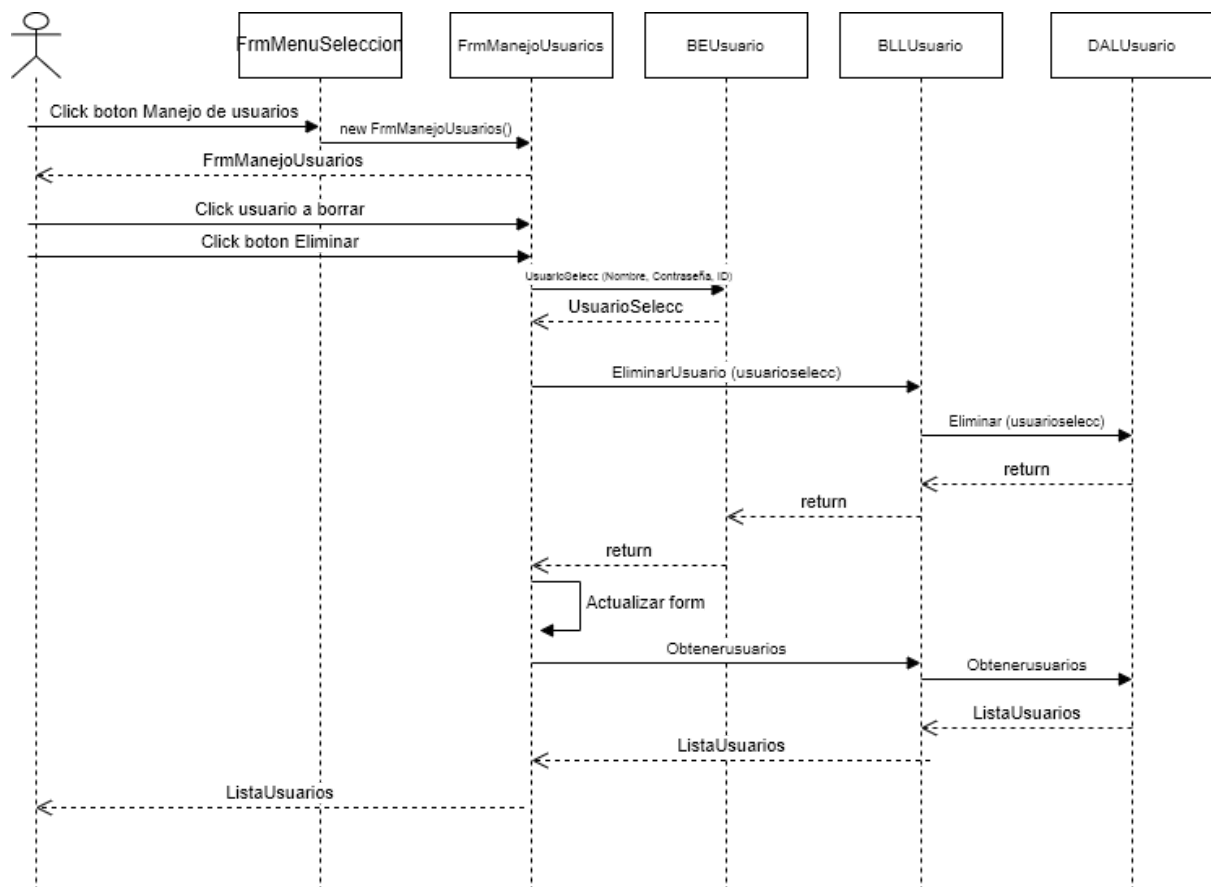


UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>			Versión: 1.0	
	<Nombre de Sección / Etapa>				

Diagrama de secuencia y especificación (Borrar)

Nombre del Caso de uso: CU005-Borrar Usuario	
Actores: Usuario	
Flujo Principal:	1-El usuario aprieta el botón “Manejo de usuarios” en el form “MenúSelección” 2-El sistema despliega el form “Manejo Usuarios” 3-El usuario hace click en el usuario a borrar en la lista 4-El usuario aprieta el botón “Borrar” 5- El sistema verifica que los datos sean correctos a-SelectedUser != Null 6-El sistema crea un usuario con los datos del seleccionado para Saber cuál borrar 7- El Sistema borra el usuario seleccionado de la lista de usuarios 8-El sistema muestra la lista de usuarios actualizada
Flujo Alternativo:	5.1-El sistema detecta que los datos no son correctos y avisa 5.2-El sistema retorna al punto 3 del flujo principal
Descripción: Se borra el usuario seleccionado de la lista de usuarios	
Precondiciones: Que el usuario haya iniciado sesión de forma correcta y tenga permisos	
Postcondiciones: Se Borra al usuario de la lista de usuarios actualizada	
Puntos de extensión: Ninguno	

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				Versión: 1.0



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				Versión: 1.0

CRIPTOGRAFIA

Objetivo

El objetivo que se quiere alcanzar con el uso de herramientas de esta naturaleza es el de poder encriptar (y, por lo tanto, hacer más seguros) los datos personales de los usuarios que se registran en el sistema, para que de esta manera, al momento de que se registran en el sistema, los datos personales no puedan ser adivinados por nadie más que no sea alguien con permisos exclusivos al sistema que estamos desarrollando (ósea, a no ser que el usuario este en la lista de usuarios permitidos del sistema, no se puede entrar y tener acceso a la lista de usuarios, mucho menos saber los datos personales de estos)

Para lograr esto hicimos uso de una clase en la capa de servicio (misma capa que contiene las sesiones con los usuarios, utilizando un IUser que viene de la capa de abstracción y cuenta con los atributos de nombre y contraseña) llamada “Encriptado”, donde tenemos un método Hash (el algoritmo que nos pareció más cómodo y adecuado para efectuar el encriptado) y 2 métodos adicionales para realizar el encriptado y desencriptado propiamente dicho (los cuales reciben texto plano, el cual en este caso serán los datos personales de los usuarios, más específicamente sus contraseñas)

De esta manera, logramos encriptar los datos de nuestros usuarios y así proteger su seguridad, además de aumentar la funcionalidad y apoyo de la capa de servicio en relación a las otras.

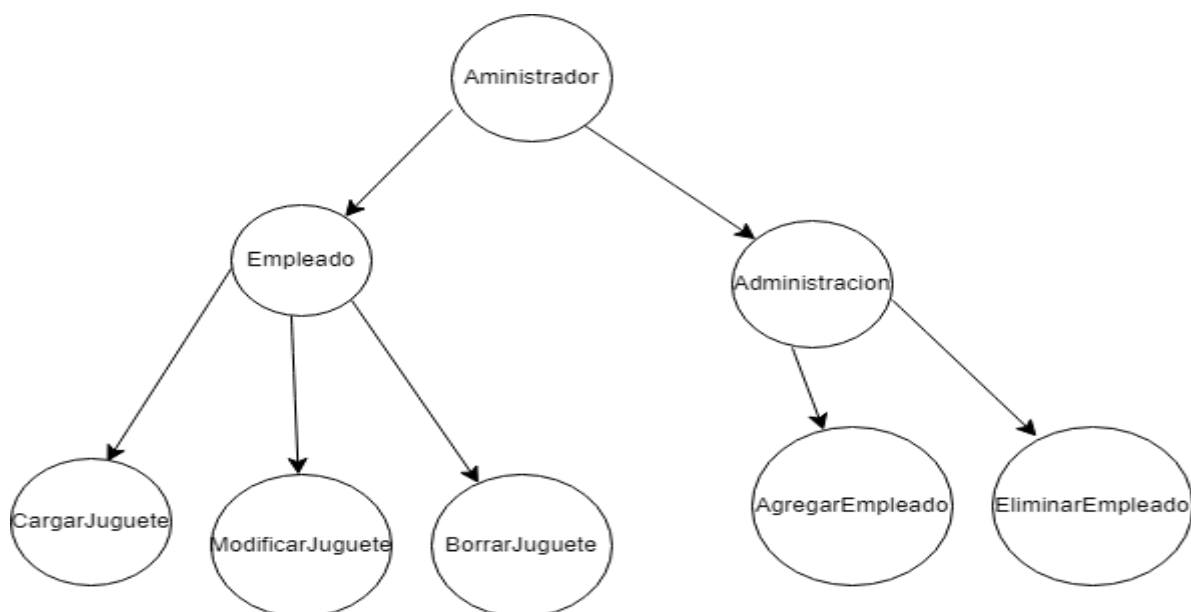
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				Versión: 1.0
	<Nombre de Sección / Etapa>				

COMPOSITE/MANEJO USUARIO

Objetivo

El objetivo que se quiere alcanzar con el uso de herramientas de esta naturaleza es el de poder tener una serie de permisos los cuales se puedan modificar y agregar (además de eliminar) a voluntad dentro del sistema, permitiendo crear permisos que permitan acceder a ciertas partes de este sistema y que puedan ser otorgados a un usuario (el cual al tener dicho permiso en su código podrá acceder a algunas partes del sistema mientras que a otras no)

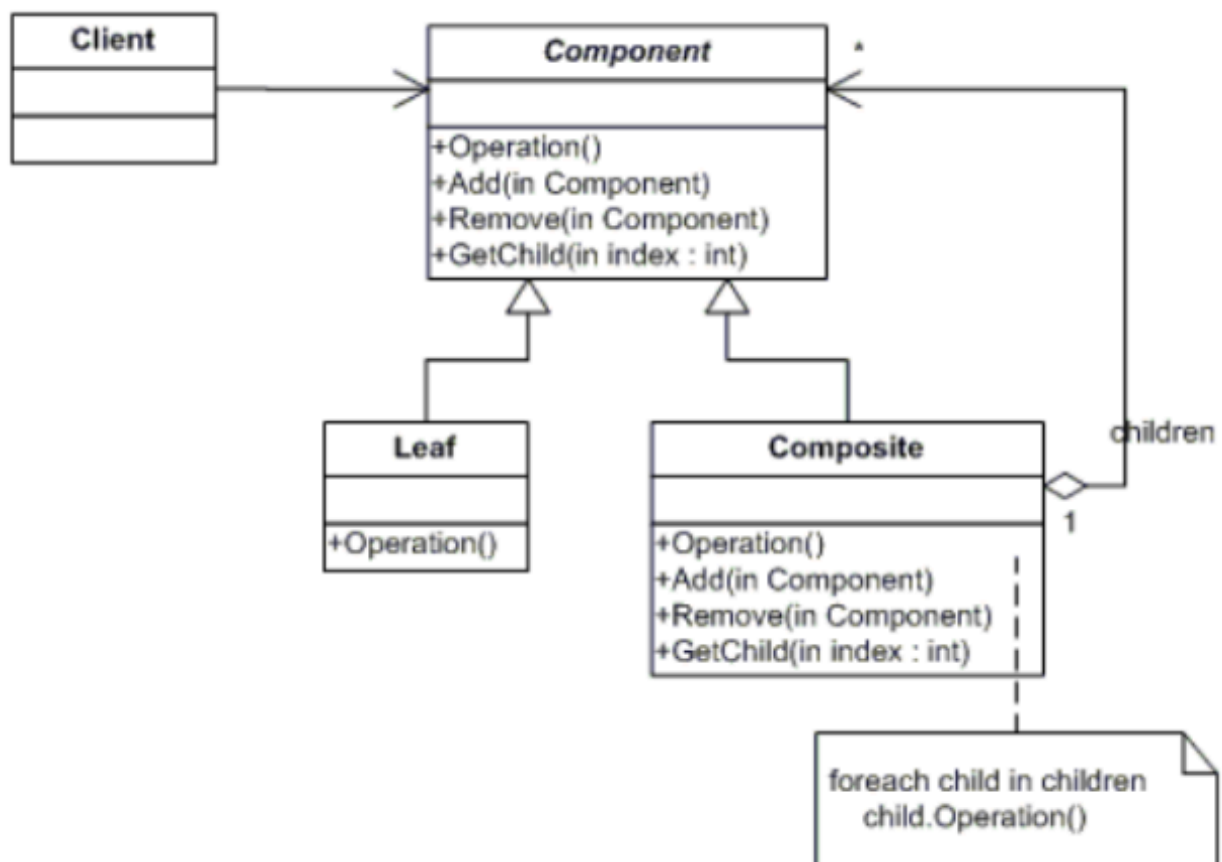
Para esto se usará el patrón composite, el cual nos permitirá representar, en forma de árbol, la relación parte-todo de la cual forman parte los distintos permisos, con cada permiso teniendo un hijo y a su vez en el caso de algunos un padre, y con algunos permisos teniendo más hijos que otros e incluyendo a una mayor cantidad de acciones, razón por la cual podrá entrar a más lugares que otros.



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				Versión: 1.0

Adicionalmente, y como se ve abajo, este patrón se encuentra representado en unas clases en específico las cuales tienen el nombre de “componente” y a su vez tienen un composite que hereda de ellos y posee la capacidad de tener hijos, formando así la estructura árbol y con el componente también pudiendo tener “hojas” las cuales, como describimos arriba, no poseen hijos

De esa forma, el diagrama general del patrón quedaría de la siguiente manera



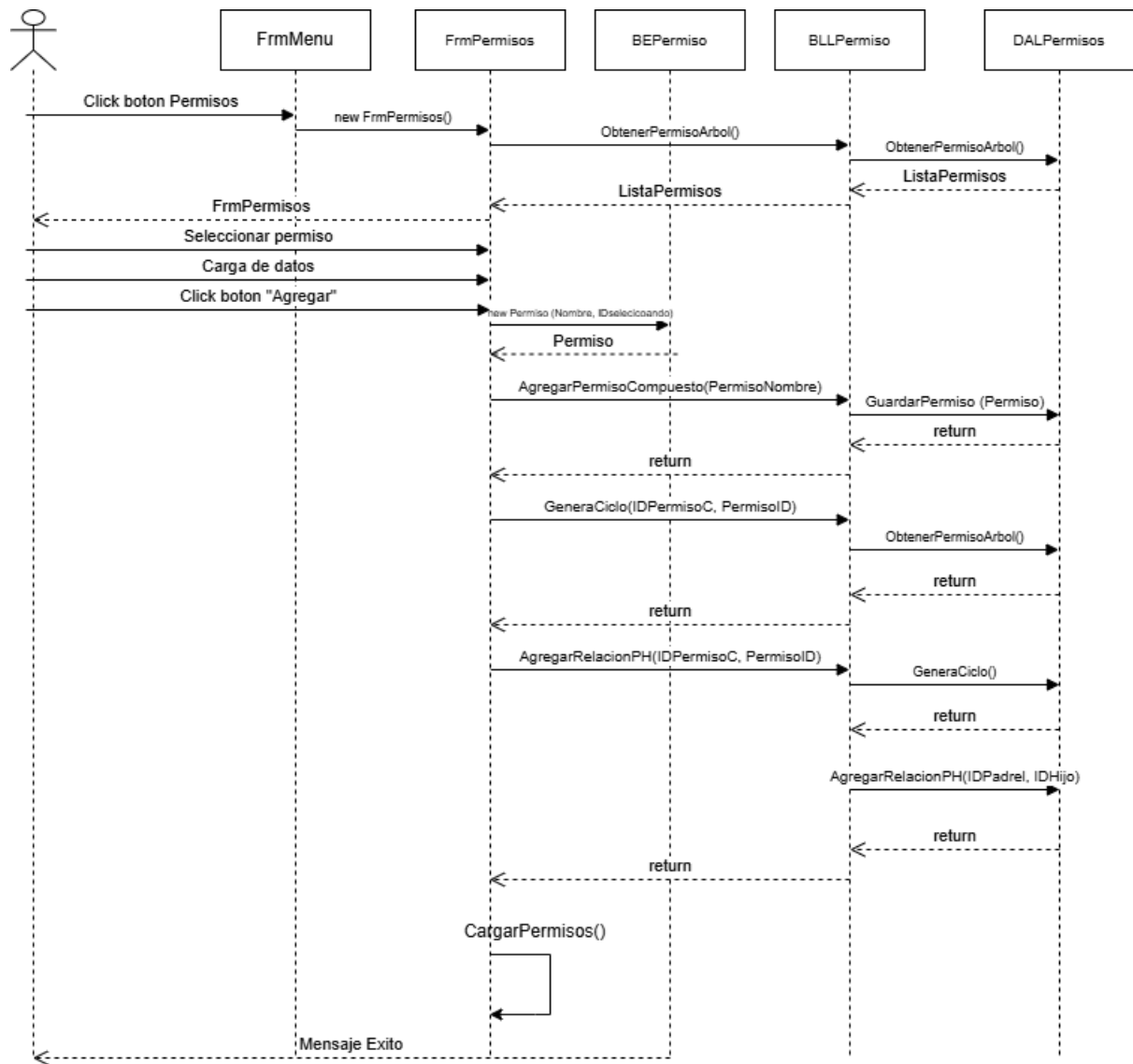
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				Versión: 1.0

Diagrama de secuencia y especificación (Agregar permiso)

Nombre del Caso de uso: CU006-Agregar Permiso	
Actores: Usuario	
Flujo Principal:	
agregar	1-El usuario aprieta el botón "Permisos" en el form "Menú" 2-El sistema despliega el form "Permisos" 3-El usuario selecciona un permiso sobre el que agregar 4-El usuario llena el textbox con el nombra del permiso a
pone al	5-El usuario aprieta el botón "Agregar" 6- El sistema verifica que los datos sean correctos a-textbox1.string != null 7-El sistema crea un permiso con el nombre introducido y
éxito	8- El Sistema guarda el permiso y manda un mensaje de
permisos	9-El sistema limpia los textbox y muestra la lista de
	Actualizada
Flujo Alternativo:	
avisa que	6.1-El sistema detecta que los datos no son correctos y Se debe escribir algo en el textbox 6.2-El sistema retorna al punto 4 del flujo principal 7.1-El sistema detecta que hubo un error al agregar el
permiso	7.2- El sistema retorna al punto 4 del flujo principal

Descripción: Se agrega un permiso a la lista de permisos disponibles como miembro de una familia
Precondiciones: Que el usuario haya iniciado sesión de forma correcta y tenga permisos
Postcondiciones: Se agrega el permiso correctamente
Puntos de extensión: Ninguno

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>			Versión: 1.0	
	<Nombre de Sección / Etapa>				



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				Versión: 1.0

Diagrama de secuencia y especificación (Eliminar permiso)

Nombre del Caso de uso: CU007-Eliminar Permiso	
Actores: Usuario	
Flujo Principal:	1-El usuario aprieta el botón “Permisos” en el form “Menú” 2-El sistema despliega el form “Permisos” 3-El usuario selecciona un permiso que desee eliminar 4-El usuario aprieta el botón “Eliminar” 5-El sistema verifica que los datos sean correctos a-permisoseleccionado != null 6- El sistema le pide al usuario que confirme si desea borrar 7-El usuario confirma su elección de borrar el permiso 8- El Sistema elimina al permiso y muestra un mensaje de éxito 9-El sistema limpia los textbox y muestra la lista de permisos Actualizada
Flujo Alternativo:	5.1-El sistema detecta que no se ha seleccionado un permiso y Pide seleccionar uno para borrar 5.2-El sistema retorna al punto 4 del flujo principal 7.1-El sistema detecta que hubo un error al eliminar el permiso 7.2- El sistema retorna al punto 3 del flujo principal
Descripción: Se elimina un permiso de la lista de permisos disponibles	
Precondiciones: Que el usuario haya iniciado sesión de forma correcta y tenga permisos	
Postcondiciones: Se elimina el permiso correctamente	
Puntos de extensión: Ninguno	

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>			Versión: 1.0	
	<Nombre de Sección / Etapa>				

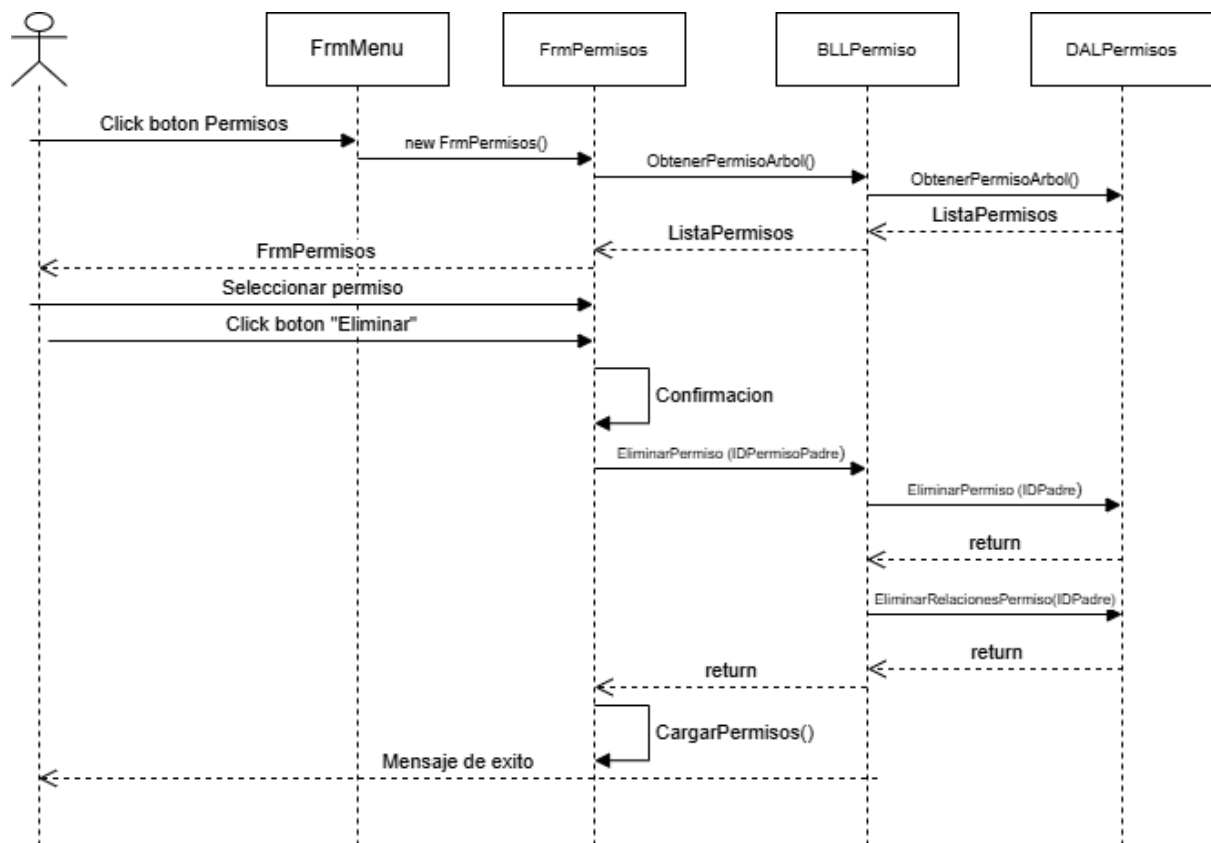
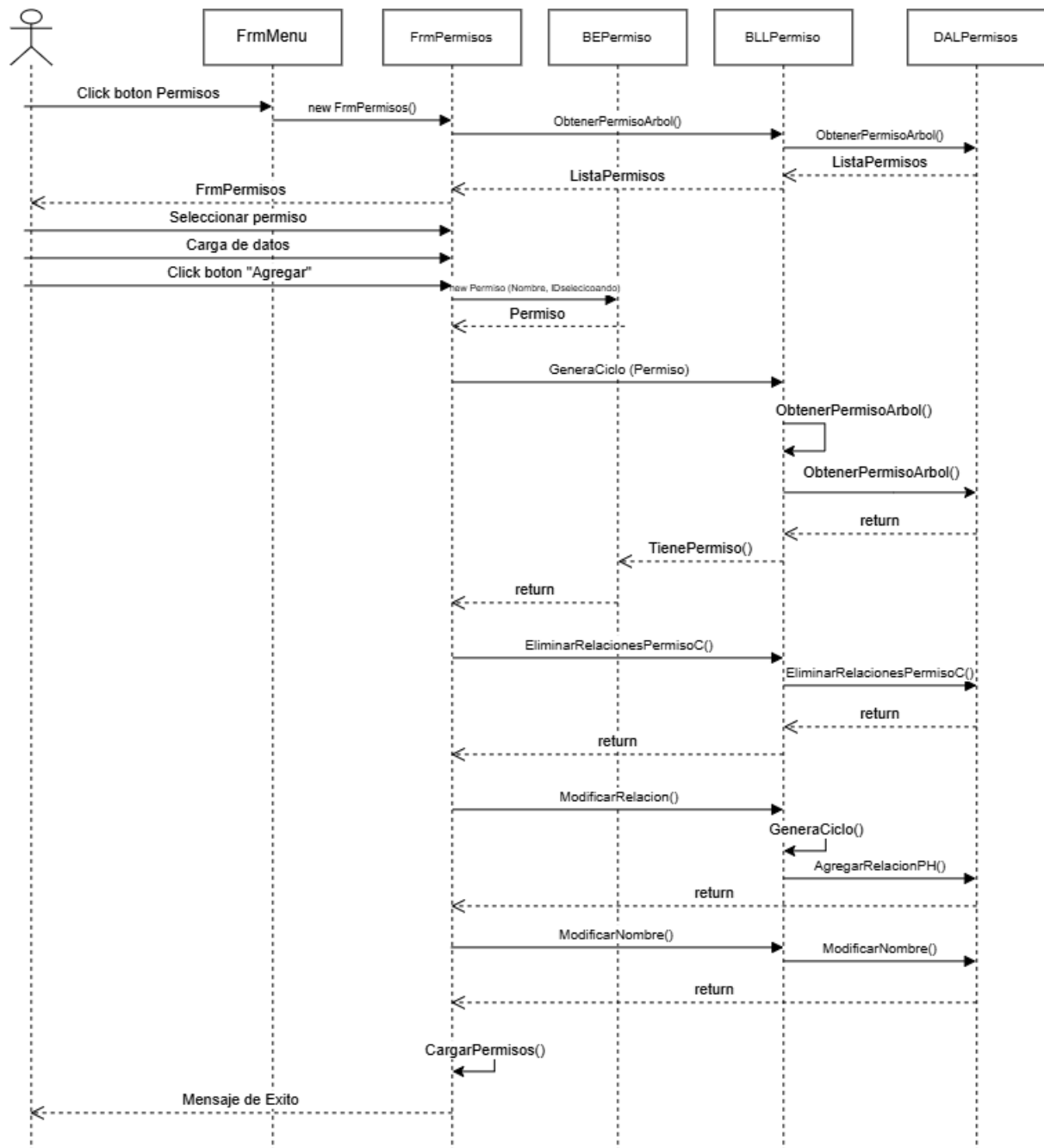


Diagrama de secuencia y especificación (Modificar Permiso)

Nombre del Caso de uso: CU008-Modificar
Actores: Usuario

Flujo Principal:	1-El usuario aprieta el botón “Permisos” en el form “Menú” 2-El sistema despliega el form “Permisos” 3-El usuario selecciona un permiso que desee Modificar 4-El usuario aprieta el botón “Eliminar” 5-El sistema verifica que los datos sean correctos a-permisoseleccionado != null 6- El sistema le pide al usuario que confirme si desea borrar 7-El usuario confirma su elección de borrar el permiso 8- El Sistema elimina al permiso y muestra un mensaje de éxito 9-El sistema limpia los textbox y muestra la lista de permisos Actualizada
Flujo Alternativo:	5.1-El sistema detecta que no se ha seleccionado un permiso y Pide seleccionar uno para borrar 5.2-El sistema retorna al punto 4 del flujo principal 7.1-El sistema detecta que hubo un error al eliminar el permiso 7.2- El sistema retorna al punto 3 del flujo principal
Descripción: Se elimina un permiso de la lista de permisos disponibles	
Precondiciones: Que el usuario haya iniciado sesión de forma correcta y tenga permisos	
Postcondiciones: Se elimina el permiso correctamente	
Puntos de extensión: Ninguno	



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>			Versión: 1.0	
	<Nombre de Sección / Etapa>				

HISTORICO DE LOGS

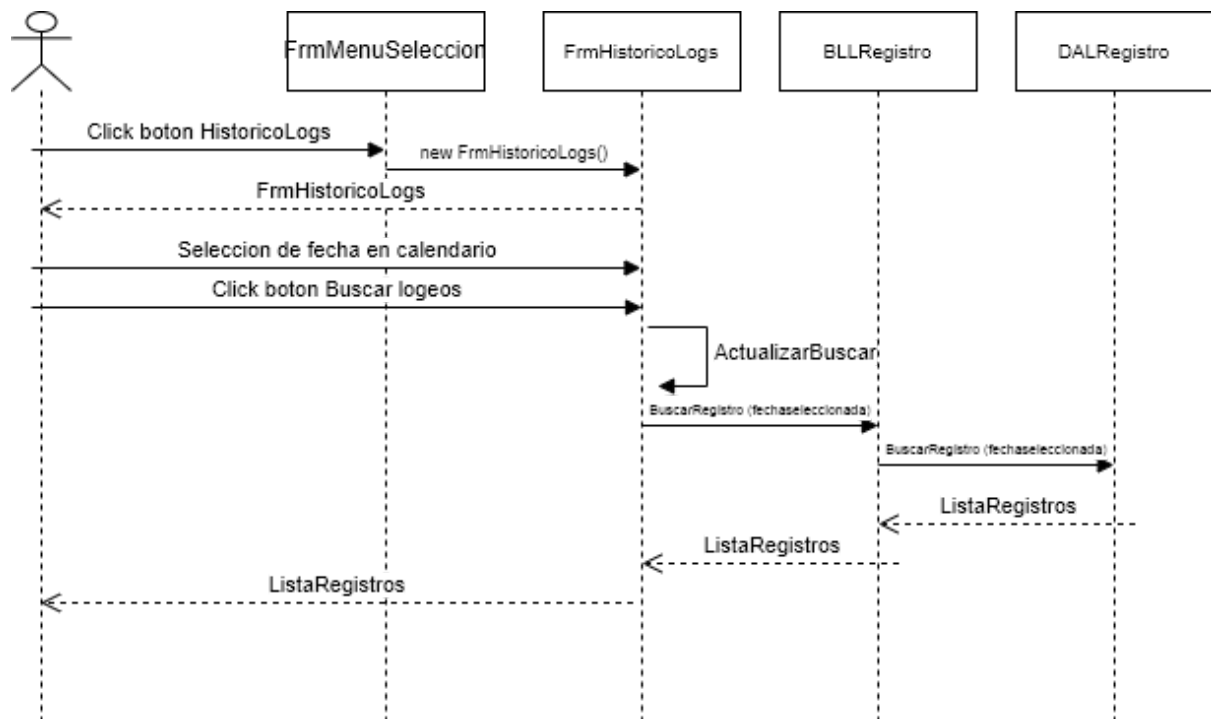
Objetivo

El objetivo en este caso es que los usuarios del sistema puedan revisar en forma de históricos los inicios de sesión de cada uno de los usuarios, así como también cuando se inició la sesión y poder buscar por fecha, permitiendo completa visibilidad y conocimiento de quien accede al sistema y cuando se hace.

Diagrama de secuencia y especificación (Buscar Historicos)

Nombre del Caso de uso: CU010-Buscar Historicos	
Actores: Usuario	
Flujo Principal:	1-El usuario aprieta el botón "HistoricosLogs" en el form "MenúSelección" 2-El sistema despliega el form "HistoricosLogs" 3-El usuario selecciona una fecha del calendario 4-El usuario hace click en buscar logeos 5- El sistema va a actualizar y busca el registro 6-El sistema devuelve la lista de registros
Flujo Alternativo:	
Descripción: Se filtra por la fecha puesta el histórico de los inicios de sesión	
Precondiciones: Que el usuario haya iniciado sesión de forma correcta y tenga permisos	
Postcondiciones: Se muestra los inicios de sesión correspondientes a la fecha indicada	
Puntos de extensión: Ninguno	

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>			Versión: 1.0	
	<Nombre de Sección / Etapa>				

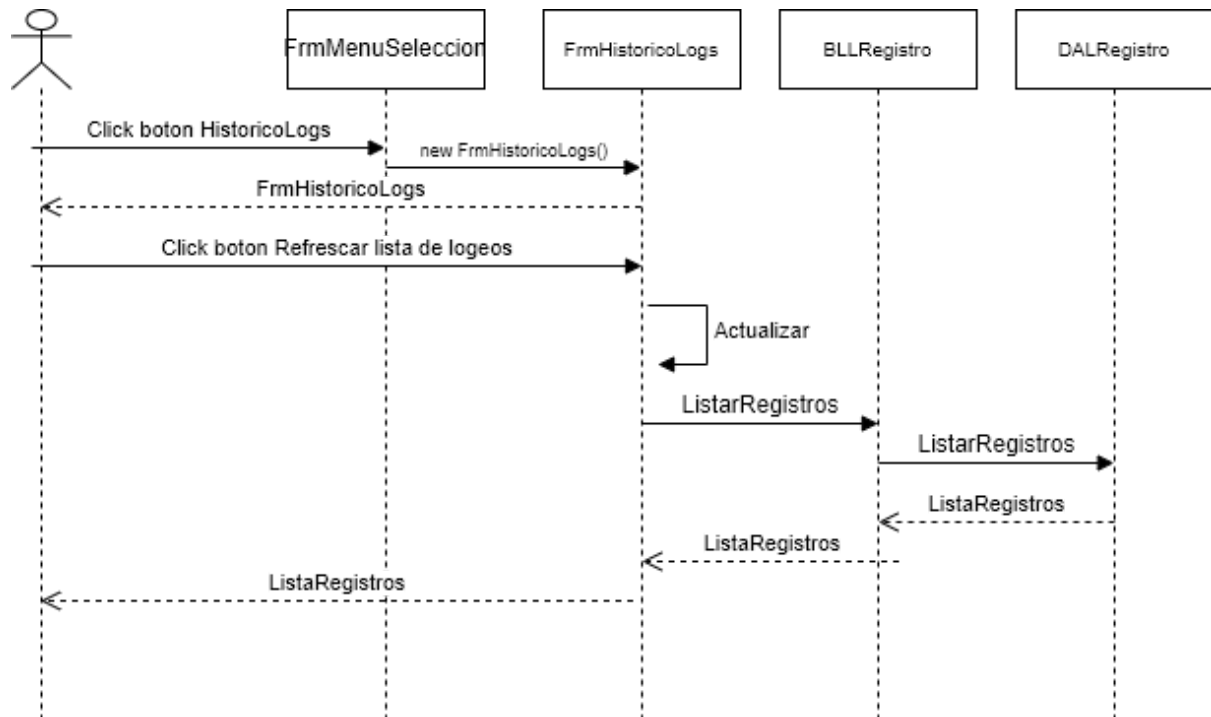


UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez	
	Alumno: Lucas Aguilar, Thiago Ragusa		Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3
	<Nombre del Sistema>			Versión: 1.0
	<Nombre de Sección / Etapa>			

Diagrama de secuencia y especificación (Refrescar Registros)

Nombre del Caso de uso: CU011-Refrescar Registros
Actores: Usuario
Flujo Principal: 1-El usuario aprieta el botón “HistoricosLogs” en el form “MenúSeleccion” 2-El sistema despliega el form “HistoricosLogs” 3-El usuario hace click en “refrescar lista de logs” 4- El sistema va a actualizar y busca la lista de registros 5-El sistema devuelve la lista de registros
Flujo alternativo
Descripción: Se refresca la lista de registros
Precondiciones: Que el usuario haya iniciado sesión de forma correcta y tenga permisos
Postcondiciones: Se muestra la lista de registros actualizada
Puntos de extensión: Ninguno

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>			Versión: 1.0	
	<Nombre de Sección / Etapa>				



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				Versión: 1.0

HISTORICO DE PRODUCTOS

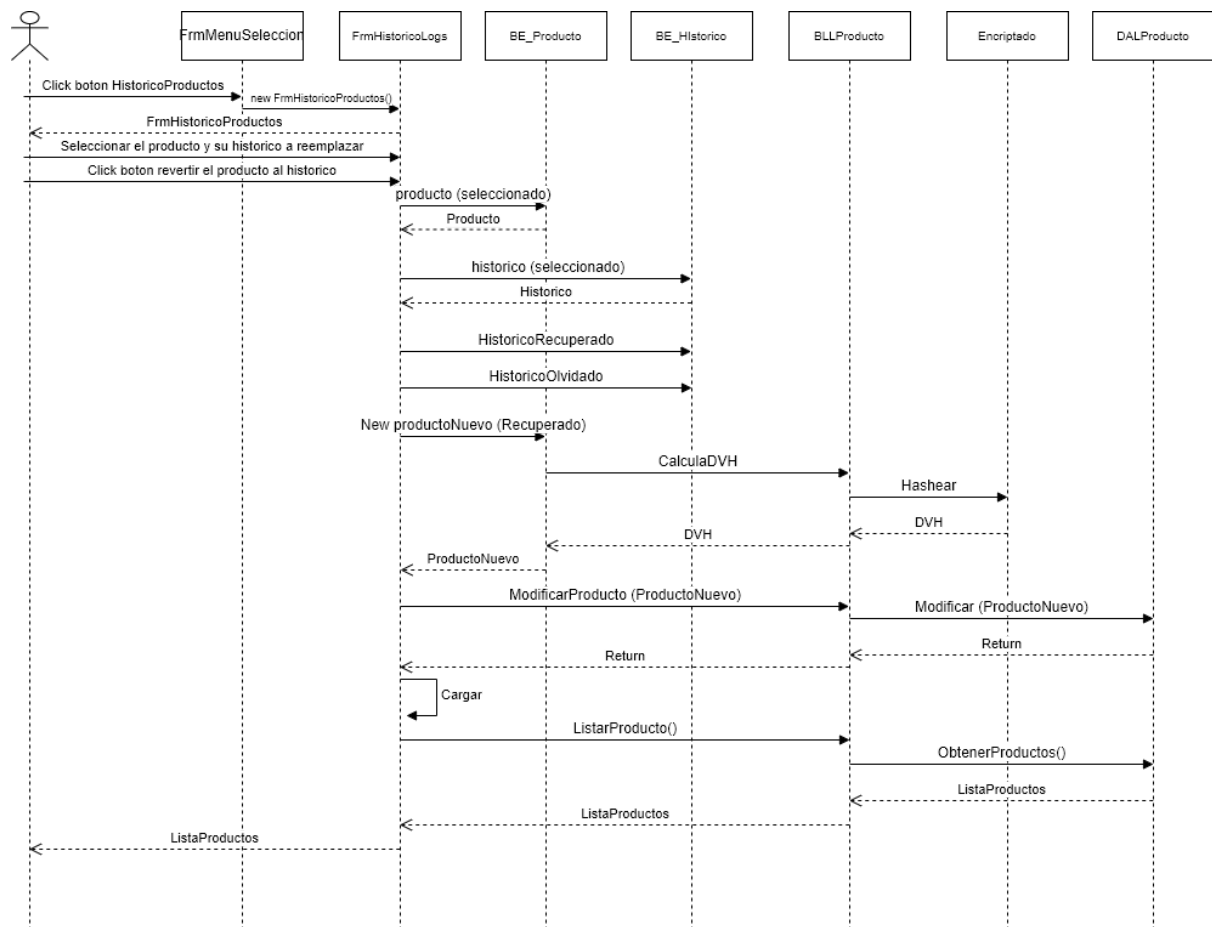
Objetivo

De la misma manera, hay un histórico de productos, permitiendo a los usuarios ver los nombres anteriores de los productos que se tienen en el sistema (un simple abm con algunos productos) y poder revertir los cambios de así quererlo

Diagrama de secuencia y especificación (Refrescar Registros)

Nombre del Caso de uso: CU012-Revertir Producto	
Actores: Usuario	
Flujo Principal:	1-El usuario aprieta el botón “Permisos” en el form “Menú” 2-El sistema despliega el form “Permisos” 3-El usuario selecciona un permiso sobre el que agregar 4-El usuario llena el textbox con el nombre del permiso a agregar 5-El usuario aprieta el botón “Agregar” 6- El sistema verifica que los datos sean correctos a-textbox1.string != null 7-El sistema crea un permiso con el nombre introducido y pone al Seleccionado como su padre 8- El Sistema guarda el permiso y manda un mensaje de éxito 9-El sistema limpia los textbox y muestra la lista de permisos Actualizada
Flujo Alternativo:	

avisa que permiso	6.1-El sistema detecta que los datos no son correctos y
	Se debe escribir algo en el textbox
	6.2-El sistema retorna al punto 4 del flujo principal
	7.1-El sistema detecta que hubo un error al agregar el
	7.2- El sistema retorna al punto 4 del flujo principal
Descripción: Se agrega un permiso a la lista de permisos disponibles como miembro de una familia	
Precondiciones: Que el usuario haya iniciado sesión de forma correcta y tenga permisos	
Postcondiciones: Se agrega el permiso correctamente	
Puntos de extensión: Ninguno	

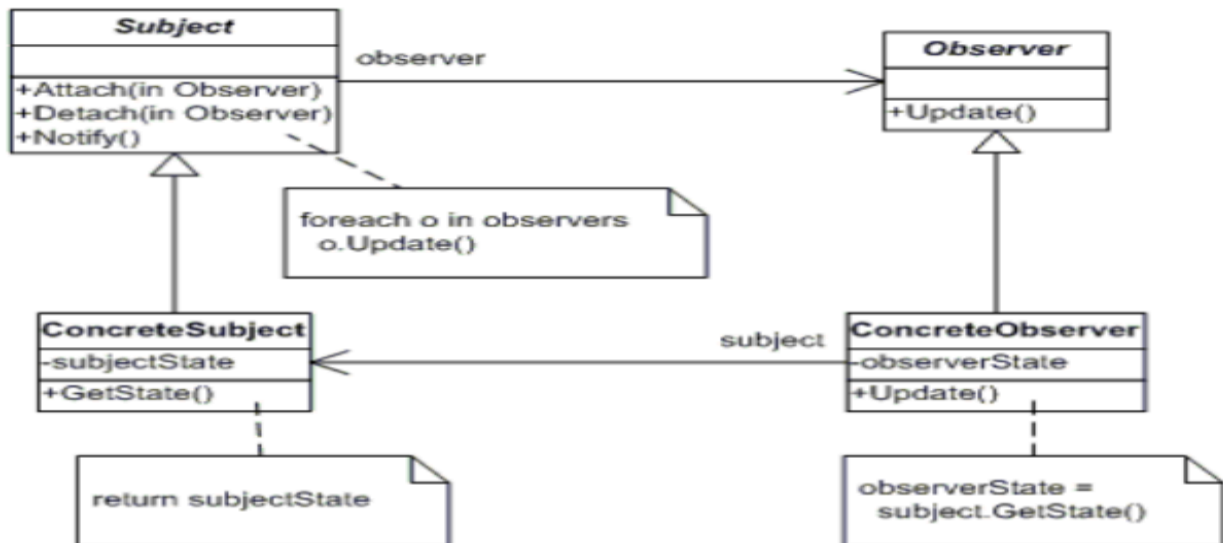


UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez	
	Alumno: Lucas Aguilar, Thiago Ragusa		Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3
	<Nombre del Sistema>			Versión: 1.0
	<Nombre de Sección / Etapa>			

OBSERVER/MULTI-IDIOMA

Objetivo

El objetivo que se quiere alcanzar con el uso de herramientas de esta naturaleza es el de poder tener la funcionalidad de multi-idioma adentro del proyecto, permitiéndonos la alteración dinámica y rápida del idioma en el cual se encuentra el programa. Dicha alteración normalmente se ejecuta por medio de las hojas de recursos (documentos los cuales contienen las traducciones de palabras a otros idiomas para luego traducir al apretar un botón por medio de un procedimiento que `c#` nos permite ejecutar. Sin embargo, en esta ocasión se optó por el patrón observer, patrón de comportamiento que define una dependencia del tipo *uno a muchos* entre objetos, de manera que cuando uno de los objetos cambia su estado, notifica este cambio a todos los dependientes, permitiéndonos igualmente cambiar el idioma en el que se encuentra un form en específico del sistema al detectarse el presionar de un botón (solo que, en vez de usar las hojas de recurso para las traducciones, se usan tablas en la base de datos)



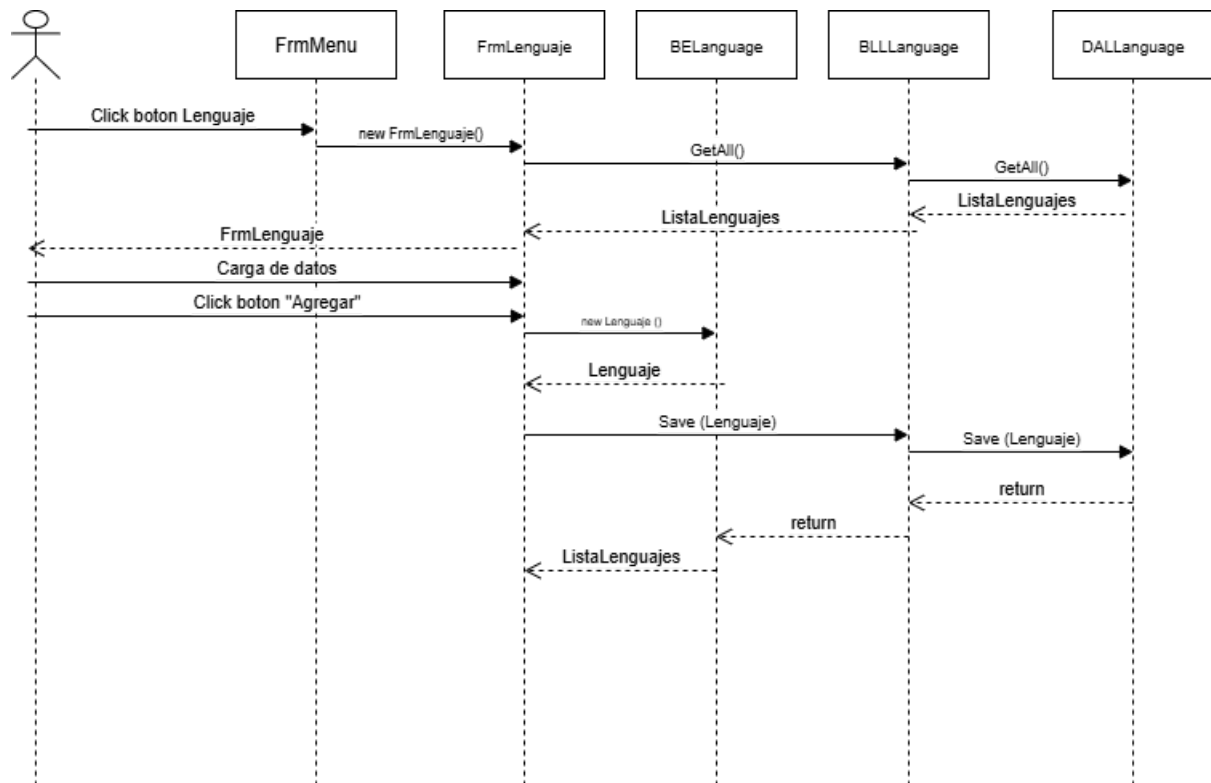
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>			Versión: 1.0	
	<Nombre de Sección / Etapa>				

Diagrama de secuencia y especificación (Agregar Lenguaje)

Nombre del Caso de uso: CU008-Agregar Lenguaje	
Actores: Usuario	
Flujo Principal:	1-El usuario aprieta el botón “Lenguaje” en el form “Menú” 2-El sistema despliega el form “Lenguaje” 3-El usuario llena el textbox con el nombra del lenguaje a agregar 4-El usuario aprieta el botón “Agregar” 5- El sistema verifica que los datos sean correctos a-textbox1.string != null 6-El sistema crea un lenguaje con el nombre introducido y lo guarda

textbox	7- El Sistema muestra un mensaje de exito y limpia el
	8-El sistema muestra la lista de lenguajes actualizada
Flujo Alternativo: avisa que se lenguaje	5.1-El sistema detecta que los datos no son correctos y Debe escribir algo en el textbox 5.2-El sistema retorna al punto 3 del flujo principal 6.1-El sistema detecta que hubo un error al agregar el 6.2- El sistema retorna al punto 3 del flujo principal
Descripción: Se agrega un lenguaje a la lista de lenguajes disponibles	
Precondiciones: Que el usuario haya iniciado sesión de forma correcta y tenga permisos	
Postcondiciones: Se agrega el lenguaje correctamente	
Puntos de extensión: Ninguno	

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez	
	Alumno: Lucas Aguilar, Thiago Ragusa		Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3
	<Nombre del Sistema>			Versión: 1.0
	<Nombre de Sección / Etapa>			



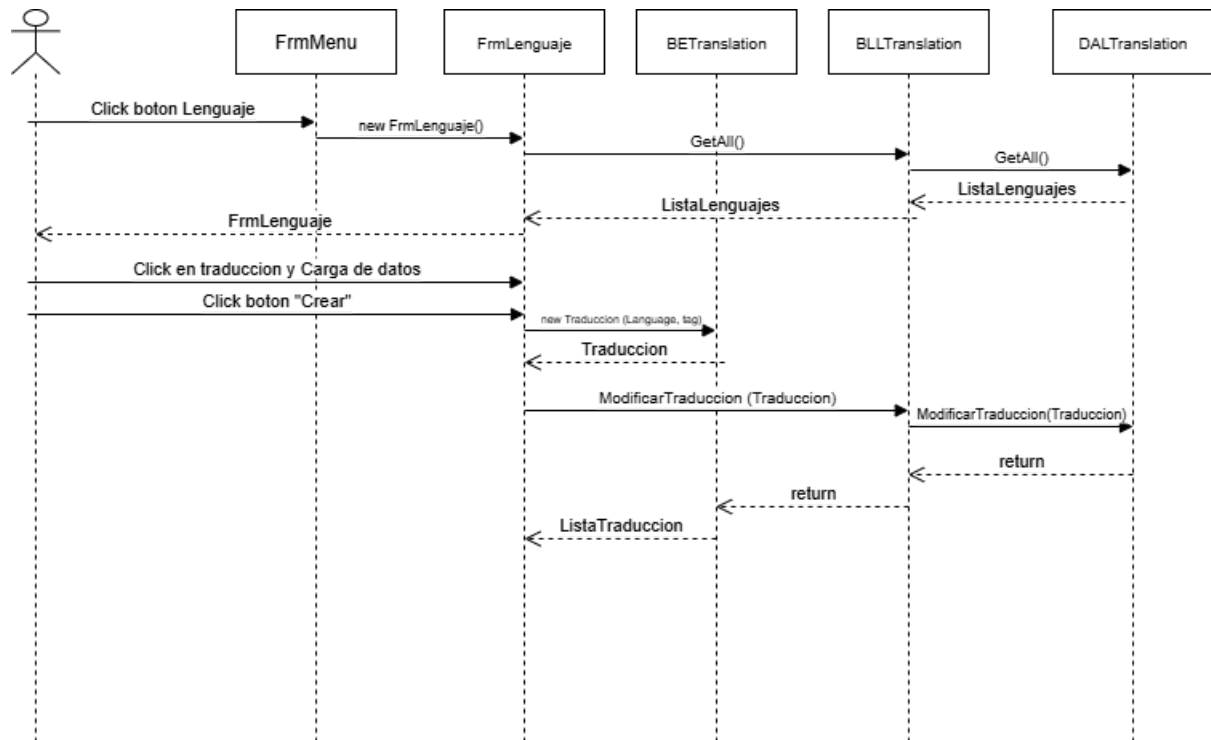
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				Versión: 1.0
	<Nombre de Sección / Etapa>				

Diagrama de secuencia y especificación (Crear Traducción)

Nombre del Caso de uso: CU009-Crear Traducción	
Actores: Usuario	
Flujo Principal:	1-El usuario aprieta el botón “Lenguaje” en el form “Menú” 2-El sistema despliega el form “Lenguaje” 3-El usuario hace click en la palabra para la que quiere crear la Traducción. 4-El usuario llena el textbox con la traducción de la palabra 5-El usuario aprieta el botón “Crear” 6- El sistema crea la traducción y la guarda 7-El sistema carga la lista de lenguajes y traducciones actualizada
Flujo Alternativo:	
Descripción:	Se crea una traducción para una palabra de un lenguaje
Precondiciones:	Que el usuario haya iniciado sesión de forma correcta y tenga permisos
Postcondiciones:	Se crea la traducción exitosamente y se muestra
Puntos de extensión:	Ninguno

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez	Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa		Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	
			Año: 3	

	<Nombre del Sistema>	Versión: 1.0
	<Nombre de Sección / Etapa>	



UNIVERSIDAD ABIERTA INTERAMERICANA			
Facultad de Tecnología Informática			
	Materia: Ingeniería de software	Docente: Julián Martín Rodríguez	Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa		

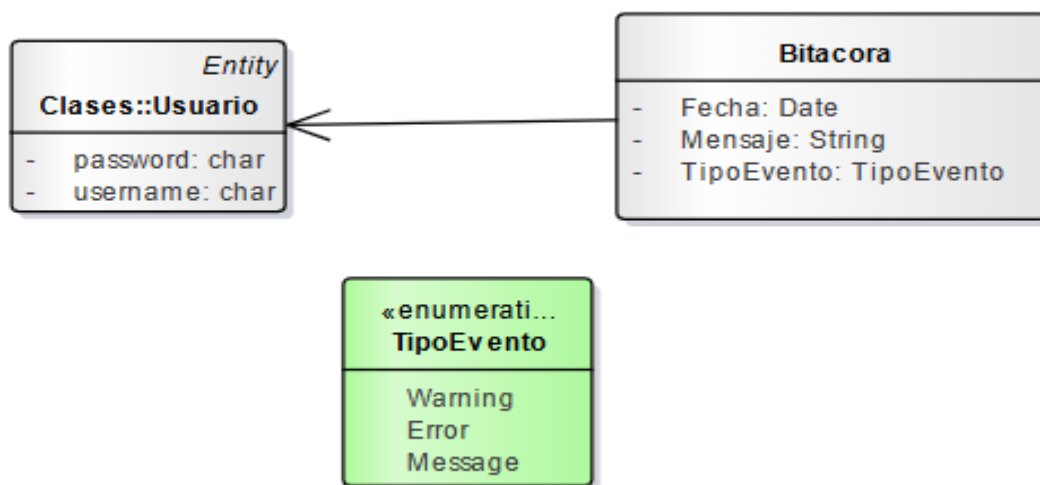
	Localización:	Comisión:	Turno:	Año:	Versión: 1.0
	Castelar	3-A	Noche	3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				

BITACORA

Objetivo

El objetivo que se quiere alcanzar al agregar una bitácora, es el de básicamente llevar un recuento de unas acciones en particular para poder ver los cambios que ocurren en relación a estas (básicamente, una forma de consulta de eventos/cambios y un registro para ellos). En el caso de nuestro trabajo, lo que se hará será llevar un registro de cuando se inicia o cierra sesión, avisando que horario ha sido el que ha hecho la acción, su ID, y avisando que tipo de acción a sido (si ha cerrado o iniciado sesión) todo esto registrándose en una tabla “bitácora” la cual guardaremos en memoria y podremos ver en el programa, con el diagrama de como funciona dicho procedimiento siendo de la siguiente manera:

class BitacoraEvento



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>			Versión: 1.0	
	<Nombre de Sección / Etapa>				

CONTROL DE CAMBIOS

Objetivo

El objetivo que se busca alcanzar al implementar el control de cambios es el de básicamente tener una bitácora interactuadle, la cual nos permita no solo ver los cambios que se han hecho sobre una entidad de una tabla (en el caso de nuestro proyecto seria la tabla de usuarios, específicamente el nombre del usuario, el cual se notificara cuando se haya cambiado por obra de algún usuario en específico) si no también revertir los cambios de dicha entidad al estado en el que estaban antes simplemente con hacer click en el registro que queramos y apretar un botón (lo cual causara que el nombre del usuario en específico sea revertido al anterior, como si se tratara de un modificar, sin causar problemas con las relaciones del usuario con sus permisos debido a que no se cambiara su ID, solo su nombre)

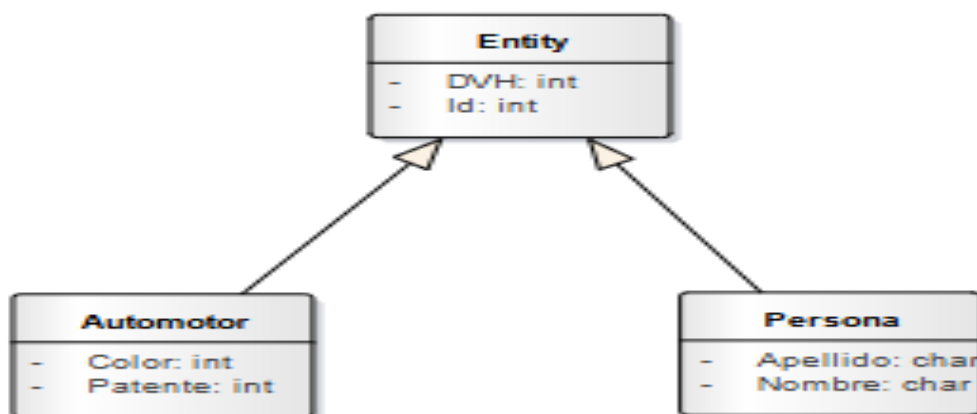
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Martín Rodríguez		Fecha: 15/4/25
	Alumno: Lucas Aguilar, Thiago Ragusa			Legajo:	
	Localización: Castelar	Comisión: 3-A	Turno: Noche	Año: 3	
	<Nombre del Sistema>				
	<Nombre de Sección / Etapa>				Versión: 1.0

DIGITO VERIFICADOR

Objetivo

El objetivo que se busca alcanzar al implementar el dígito verificador en nuestro trabajo es básicamente el de tener una manera de asegurar la integridad de los datos y, en caso de que ocurra un error en la ejecución o traslado de la información de la base de datos (por ejemplo, si se hashea un campo de los que estamos usando), se puede descubrir si se modificó el dato y que tipo de error ocurrió sin cortar tan abruptamente el programa. En este caso, lo haremos de igual forma con los usuarios, haciendo que como se ve en el diagrama estos desciendan de cierta manera de una entidad con un dígito verificador el cual se le asignara a cada uno de estos para poder usarlo en caso de error, como se ve abajo

class Ejemplos



Aunque este tipo de dígitos también se puede hacer por entidad en vez de registro como se ve arriba

	DVV
1	265754791